PYTHON ACTIVITY-BASED LEARNING REPORT

Name: P. Hemanth	**Roll No:** 26000187
Branch: CSE	**Course:** Python Programming

Case 1: Numeric Pattern Generator

Activity list reference: Case 27 from activity list

Problem Statement

Write a Python program to generate a right-angled number pattern for a user-specified number of rows.

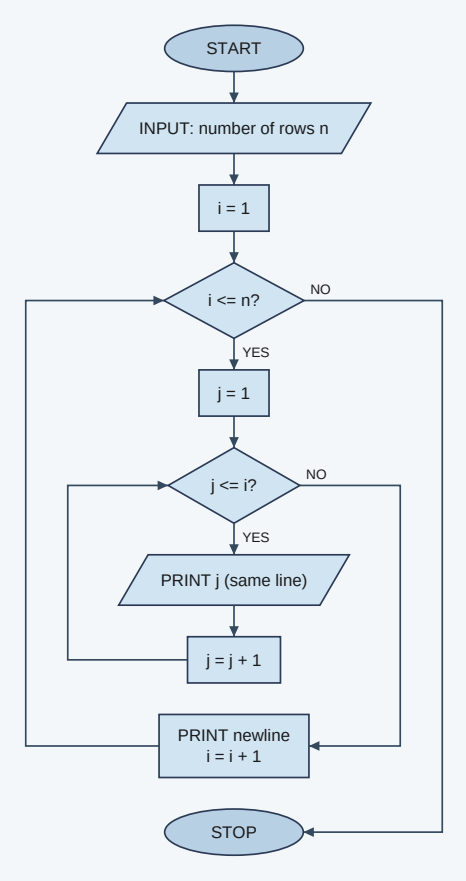
Objectives and Assumptions

Objective: Generate rows of numbers using nested loops.
Assumption: The number of rows is a positive integer.

Algorithm

- Step 1: Start.
- Step 2: Read the number of rows, n.
- Step 3: Repeat i from 1 to n.
- Step 4: For each row, print numbers from 1 to i.
- Step 5: Move to the next line and stop after n rows.

Flowchart



Pseudocode

```
READ n
FOR i = 1 TO n
    FOR j = 1 TO i
        PRINT j without newline
    PRINT newline
STOP
```

Source Code & Output

```
n = int(input("Enter number of rows: "))
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

```
Enter number of rows: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Test Case and Conclusion

Test: Input: 3 | Expected pattern has 3 rows.
Conclusion: The program uses nested loops to generate the required number pattern.

Case 2: Password Policy Validation

Activity list reference: Case 29 from activity list

Problem Statement

Write a Python program to check whether a sample password meets minimum length, letter, and digit requirements.

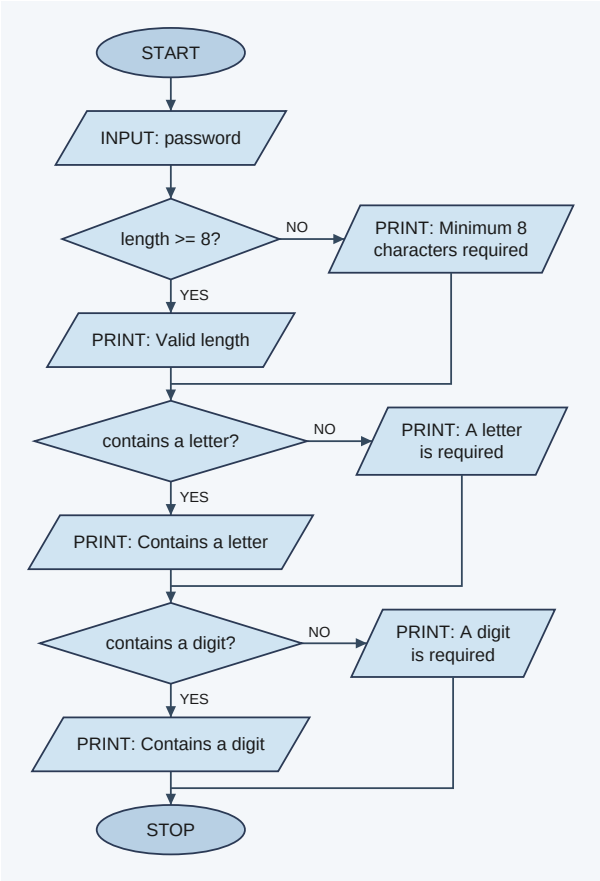
Objectives and Assumptions

Objective: Validate a password using string operations and conditional statements.
Assumption: A valid sample password must contain at least 8 characters, one letter, and one digit.

Algorithm

- Step 1: Start.
- Step 2: Read a password string.
- Step 3: Check whether its length is at least 8.
- Step 4: Check whether it contains a letter and a digit.
- Step 5: Display each validation result and stop.

Flowchart



Pseudocode

```
READ password
CHECK length >= 8
CHECK at least one letter
CHECK at least one digit
DISPLAY validation results
STOP
```

Source Code & Output

```
password = input("Enter password: ")
if len(password) >= 8:
    print("Valid length")
else:
    print("Minimum 8 characters required")
if any(ch.isalpha() for ch in password):
    print("Contains a letter")
else:
    print("A letter is required")
if any(ch.isdigit() for ch in password):
    print("Contains a digit")
else:
    print("A digit is required")
```

```
Enter password: hello123
Valid length
Contains a letter
Contains a digit
```

Test Case and Conclusion

Test: Input: abc123 | Expected: length requirement is not met.
Conclusion: The program validates basic password rules using strings and if-else statements.

Case 3: Text Compression Using Character Runs

Activity list reference: Case 35 from activity list

Problem Statement

Write a Python program to group consecutive repeated characters and represent each group by its character and count.

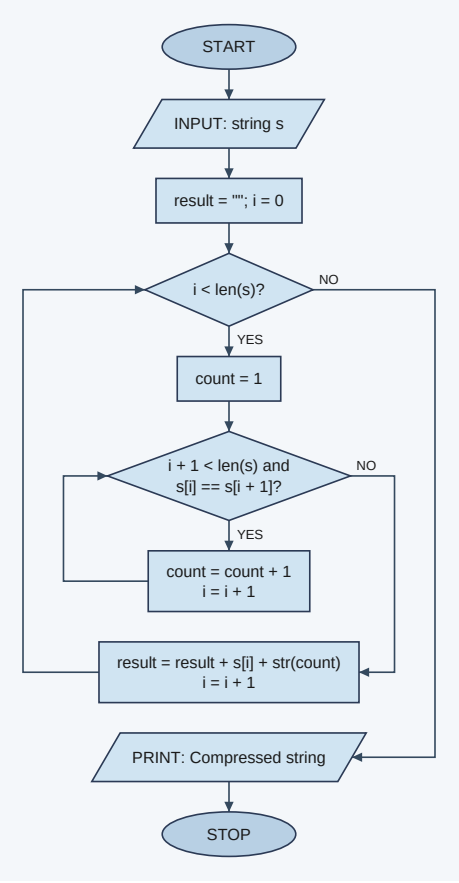
Objectives and Assumptions

Objective: Compress repeated characters using string traversal and loops.
Assumption: The input is a non-empty string; each character group is encoded as character followed by count.

Algorithm

- Step 1: Start.
- Step 2: Read a string.
- Step 3: Set an index to 0 and an empty result string.
- Step 4: Count consecutive matching characters.
- Step 5: Append the character and its count to the result.
- Step 6: Display the compressed string and stop.

Flowchart



Pseudocode

```
READ s; result = empty; i = 0
WHILE i < length(s)
    COUNT consecutive equal characters
    APPEND character and count to result
    MOVE i forward
DISPLAY result
STOP
```

Source Code & Output

```
s = input("Enter a string: ")
result = ""
i = 0
while i < len(s):
    count = 1
    while i + 1 < len(s) and s[i] == s[i + 1]:
        count += 1
        i += 1
    result += s[i] + str(count)
    i += 1
print("Compressed string:", result)
```

```
Enter a string: aaabbcccc
Compressed string: a3b2c4
```

Test Case and Conclusion

Test: Input: aabb | Expected compressed string: a2b2.
Conclusion: The program counts consecutive character groups and creates a compressed representation.

Case 4: Repeated Word Detection

Activity list reference: Case 36 from activity list

Problem Statement

Write a Python program to identify words that occur more than once in a sentence.

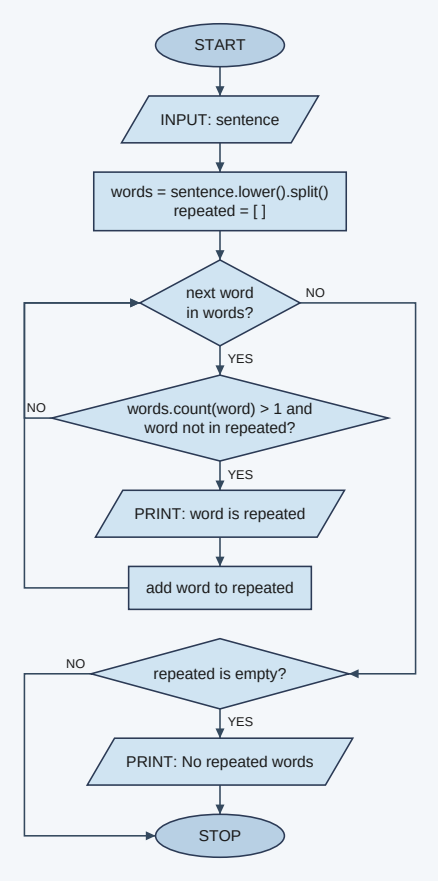
Objectives and Assumptions

Objective: Use string splitting and list operations to find repeated words.
Assumption: Words are separated by spaces; comparison is case-insensitive.

Algorithm

- Step 1: Start.
- Step 2: Read a sentence and convert it to lowercase.
- Step 3: Split the sentence into a list of words.
- Step 4: Visit each word and check its frequency.
- Step 5: Print each repeated word only once.
- Step 6: Stop.

Flowchart



Pseudocode

```
READ sentence
words = lowercase sentence split by spaces
FOR each word in words
    IF frequency(word) > 1 AND not already displayed
        PRINT word is repeated
IF no word was repeated: PRINT "No repeated words"
STOP
```

Source Code & Output

```
sentence = input("Enter a sentence: ").lower()
words = sentence.split()
repeated = []
for word in words:
    if words.count(word) > 1 and word not in repeated:
        print(word, "is repeated")
        repeated.append(word)
if len(repeated) == 0:
    print("No repeated words")
```

```
Enter a sentence: hello world hello python
hello is repeated
```

Test Case and Conclusion

Test: Input: red blue red | Expected: red is repeated.
Conclusion: The program uses strings and lists to detect repeated words without printing duplicates.

Case 5: Nested Loop Calendar Generator

Activity list reference: Case 37 from activity list

Problem Statement

Write a Python program to display the dates of a month in calendar form using the starting weekday and number of days.

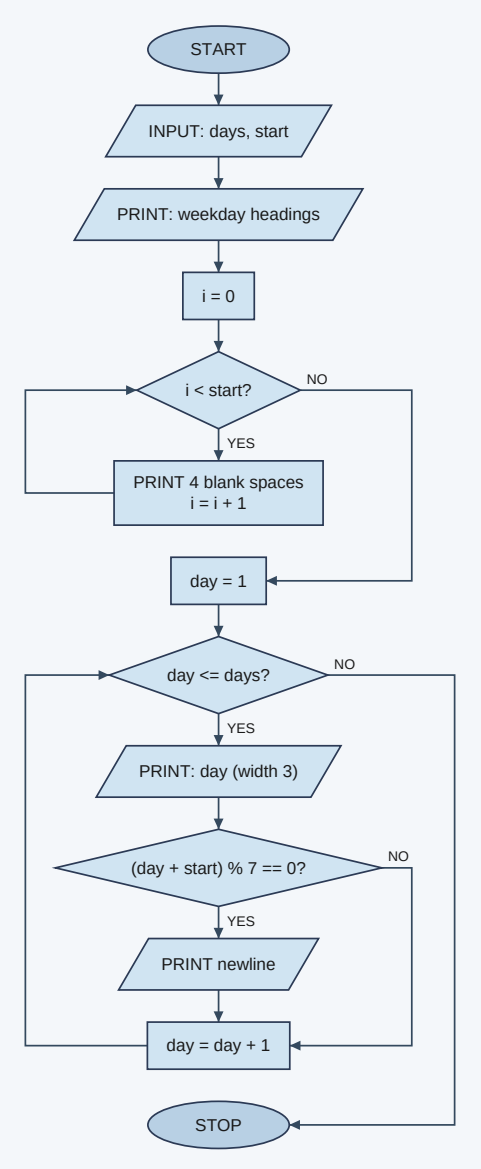
Objectives and Assumptions

Objective: Arrange date numbers into weeks using loops.
Assumption: Weekday is entered as 0 for Monday through 6 for Sunday; days is between 1 and 31.

Algorithm

- Step 1: Start.
- Step 2: Read number of days and starting weekday.
- Step 3: Print weekday headings.
- Step 4: Print blank spaces for days before the first date.
- Step 5: Print dates from 1 to the given number of days.
- Step 6: Start a new line after every Sunday and stop.

Flowchart



Pseudocode

```
READ days, start
PRINT weekday headings
PRINT start blank spaces
FOR day = 1 TO days
    PRINT day
    IF (day + start) is divisible by 7: NEW LINE
STOP
```

Source Code & Output

```
days = int(input("Enter number of days: "))
start = int(input("Starting weekday (0=Mon): "))
print("Mon Tue Wed Thu Fri Sat Sun")
for i in range(start):
    print("    ", end="")
for day in range(1, days + 1):
    print("%3d " % day, end="")
    if (day + start) % 7 == 0:
        print()
```

```
Enter number of days: 7
Starting weekday (0=Mon): 0
Mon Tue Wed Thu Fri Sat Sun
  1   2   3   4   5   6   7
```

Test Case and Conclusion

Test: Input: days=5, start=2 | Expected: first date appears under Wednesday.
Conclusion: The program uses loops and simple arithmetic to format a monthly calendar.

Case 6: Student Marks Matrix Analysis

Activity list reference: Case 42 from activity list

Problem Statement

Write a Python program to calculate the total and average marks of each student stored in a nested list.

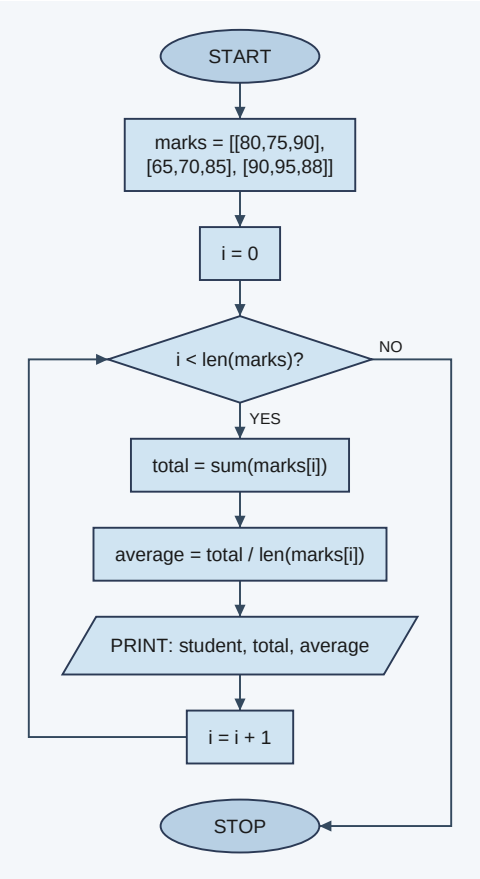
Objectives and Assumptions

Objective: Process student marks using a two-dimensional list.
Assumption: Each row contains the marks of one student and all students have the same number of subjects.

Algorithm

- Step 1: Start.
- Step 2: Create a nested list containing student marks.
- Step 3: Visit each student's row.
- Step 4: Calculate the total and average marks.
- Step 5: Display the result for each student and stop.

Flowchart



Pseudocode

```
CREATE marks matrix
FOR each student row
    total = SUM(row)
    average = total / number of subjects
    DISPLAY total and average
STOP
```

Source Code & Output

```
marks = [
    [80, 75, 90],
    [65, 70, 85],
    [90, 95, 88]
]
for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])
    print("Student", i + 1)
    print("Total:", total)
    print("Average:", round(average, 2))
```

```
Student 1
Total: 245
Average: 81.67
Student 2
Total: 220
Average: 73.33
Student 3
Total: 273
Average: 91.0
```

Test Case and Conclusion

Test: Student marks: [10, 20, 30] | Expected total = 60 and average = 20.
Conclusion: The program uses nested lists and loops to calculate student-wise statistics.

Case 7: Weekly Electricity Consumption Matrix

Activity list reference: Case 45 from activity list

Problem Statement

Write a Python program to calculate weekly electricity consumption for appliances using readings recorded over seven days.

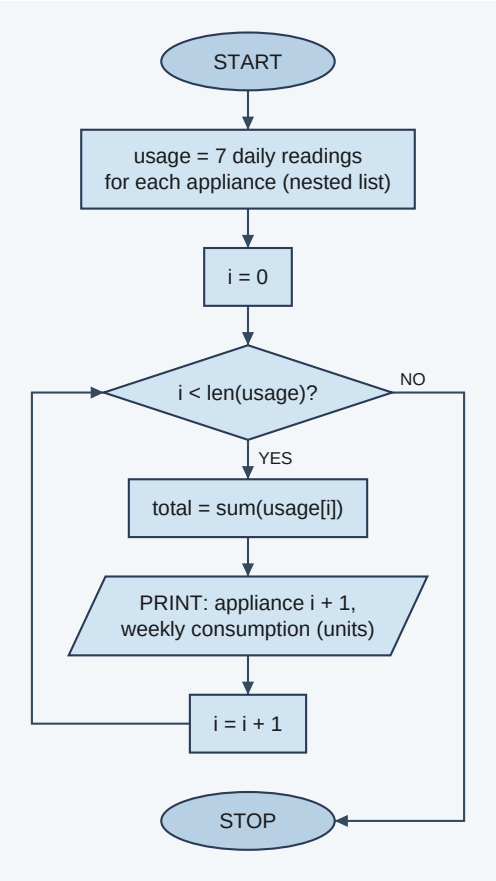
Objectives and Assumptions

Objective: Calculate appliance-wise weekly totals using nested lists.
Assumption: Each nested list row represents one appliance and contains seven daily readings in units.

Algorithm

- Step 1: Start.
- Step 2: Store seven daily readings for each appliance in a nested list.
- Step 3: Visit each appliance row.
- Step 4: Calculate the sum of its seven readings.
- Step 5: Display weekly consumption and stop.

Flowchart



Pseudocode

```
CREATE usage matrix
FOR each appliance row
    total = SUM(row)
    DISPLAY appliance and total units
STOP
```

Source Code & Output

```
usage = [
    [10, 12, 15, 10, 14, 13, 12],
    [5, 6, 7, 5, 6, 8, 7]
]
for i in range(len(usage)):
    total = sum(usage[i])
    print("Appliance", i + 1)
    print("Weekly consumption:", total, "units")
```

```
Appliance 1
Weekly consumption: 86 units
Appliance 2
Weekly consumption: 44 units
```

Test Case and Conclusion

Test: Readings [1, 1, 1, 1, 1, 1, 1] | Expected weekly total = 7 units.
Conclusion: The program uses nested lists and sum() to calculate weekly appliance consumption.

Case 8: Tuple-Based Product Catalogue Audit

Activity list reference: Case 49 from activity list

Problem Statement

Write a Python program to store product details in tuples, calculate inventory value, and identify products that need replenishment.

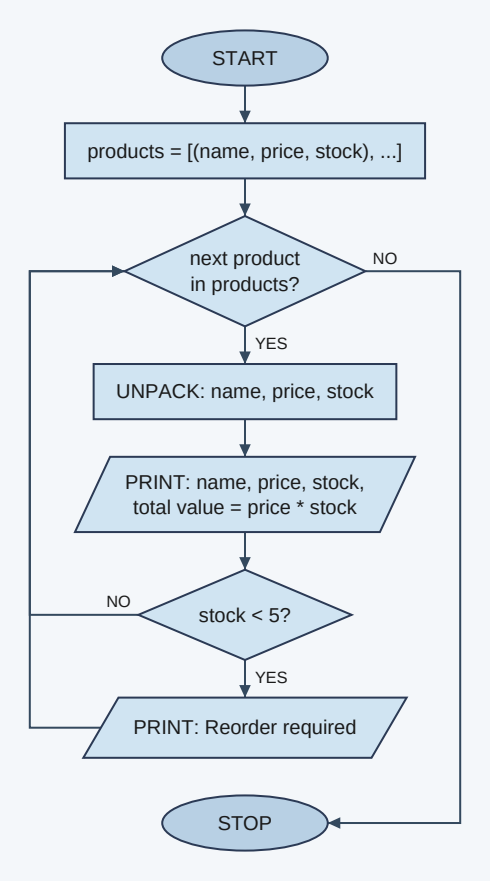
Objectives and Assumptions

Objective: Use tuples and tuple unpacking to audit product stock.
Assumption: Each product tuple stores (name, price, stock); stock below 5 requires replenishment.

Algorithm

- Step 1: Start.
- Step 2: Create a list of product tuples.
- Step 3: Unpack name, price, and stock for each product.
- Step 4: Calculate inventory value as price multiplied by stock.
- Step 5: If stock is below 5, display reorder required.
- Step 6: Stop.

Flowchart



Pseudocode

```
CREATE product tuples
FOR each tuple
    UNPACK name, price, stock
    value = price * stock
    DISPLAY product details and value
    IF stock < 5: DISPLAY "Reorder required"
STOP
```

Source Code & Output

```
products = [
    ("Laptop", 50000, 5),
    ("Mouse", 500, 10),
    ("Keyboard", 1000, 2)
]
for name, price, stock in products:
    print("Product:", name)
    print("Price:", price)
    print("Stock:", stock)
    print("Total value:", price * stock)
    if stock < 5:
        print("Reorder required")
    print()
```

```
Product: Laptop
Price: 50000
Stock: 5
Total value: 250000

Product: Mouse
Price: 500
Stock: 10
Total value: 5000

Product: Keyboard
Price: 1000
Stock: 2
Total value: 2000
Reorder required
```

Test Case and Conclusion

Test: Product tuple: (Pen, 10, 2) | Expected value = 20 and reorder required.
Conclusion: The program demonstrates tuples, unpacking, calculations, and stock validation.

Case 9: Food Waste Monitoring

Activity list reference: Case 53 from activity list

Problem Statement

Write a Python program to calculate the amount and percentage of food wasted over several days.

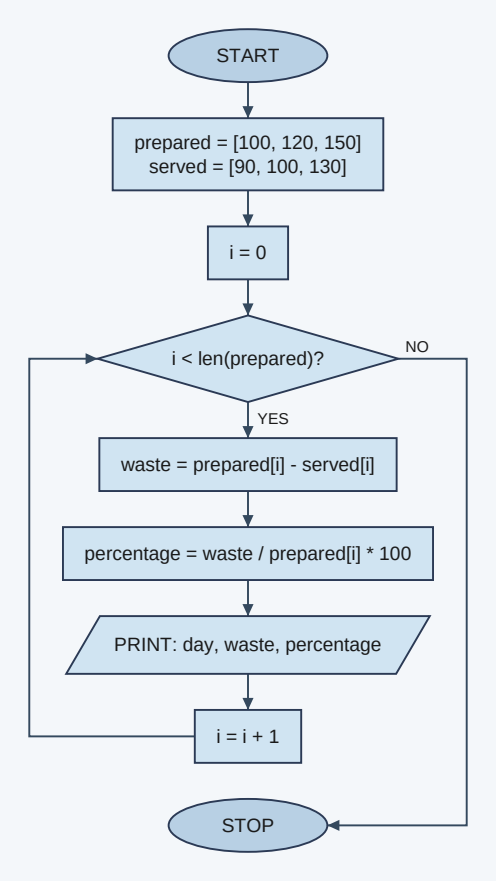
Objectives and Assumptions

Objective: Compare prepared meals with served meals and calculate waste percentages.
Assumption: Prepared quantity is greater than or equal to served quantity for each day.

Algorithm

- Step 1: Start.
- Step 2: Store prepared and served quantities in lists.
- Step 3: Repeat for each day.
- Step 4: Calculate waste = prepared minus served.
- Step 5: Calculate waste percentage.
- Step 6: Display daily waste and percentage; stop.

Flowchart



Pseudocode

```
STORE prepared[] and served[]
FOR each day i
    waste = prepared[i] - served[i]
    percentage = waste / prepared[i] * 100
    DISPLAY waste and percentage
STOP
```

Source Code & Output

```
prepared = [100, 120, 150]
served = [90, 100, 130]
for i in range(len(prepared)):
    waste = prepared[i] - served[i]
    percentage = (waste / prepared[i]) * 100
    print("Day", i + 1)
    print("Food wasted:", waste)
    print("Waste percentage:", round(percentage, 2), "%")
```

```
Day 1
Food wasted: 10
Waste percentage: 10.0 %
Day 2
Food wasted: 20
Waste percentage: 16.67 %
Day 3
Food wasted: 20
Waste percentage: 13.33 %
```

Test Case and Conclusion

Test: Prepared = 50, served = 45 | Expected waste = 5 and waste percentage = 10%.
Conclusion: The program uses lists, loops, and percentage calculations to monitor food waste.

Case 10: Multi-Product Price Fluctuation Analysis

Activity list reference: Case 63 from activity list

Problem Statement

Write a Python program to compare old and new prices of products and calculate the percentage change.

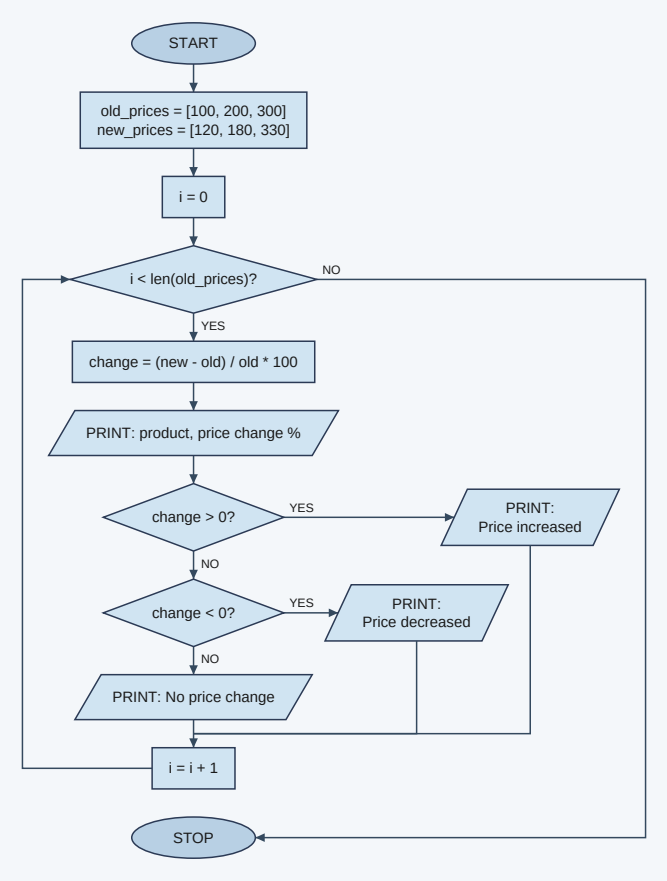
Objectives and Assumptions

Objective: Identify price increases and decreases using lists and conditional statements.
Assumption: Old prices are positive and both lists contain the same number of products.

Algorithm

- Step 1: Start.
- Step 2: Store old and new prices in two lists.
- Step 3: Visit each product.
- Step 4: Calculate percentage change from old price to new price.
- Step 5: Classify the change as increased, decreased, or unchanged.
- Step 6: Display the result and stop.

Flowchart



Pseudocode

```
STORE old_prices[] and new_prices[]
FOR each product i
    change = (new - old) / old * 100
    IF change > 0: DISPLAY increased
    ELSE IF change < 0: DISPLAY decreased
    ELSE: DISPLAY no price change
STOP
```

Source Code & Output

```
old_prices = [100, 200, 300]
new_prices = [120, 180, 330]
for i in range(len(old_prices)):
    change = ((new_prices[i] - old_prices[i])
              / old_prices[i]) * 100
    print("Product", i + 1)
    print("Price change:", round(change, 2), "%")
    if change > 0:
        print("Price increased")
    elif change < 0:
        print("Price decreased")
    else:
        print("No price change")
```

```
Product 1
Price change: 20.0 %
Price increased
Product 2
Price change: -10.0 %
Price decreased
Product 3
Price change: 10.0 %
Price increased
```

Test Case and Conclusion

Test: Old = 100, new = 100 | Expected change = 0% and no price change.
Conclusion: The program calculates percentage price changes and classifies each product.